OPERATING SYSTEM DESIGN

S Thenmozhi

Department of Computer Applications

OPERATING SYSTEM DESIGN

PROCESS & THREAD MANAGMENT

S Thenmozhi

Department of Computer Applications

Semaphores

- Semaphore(S) is a counter that is manipulated atomically through two operations : **Signal and Wait.**
- **Wait: decrement if counter is zero** then block until the semaphore is signaled
- **Signal: increment counter**, wakeup one waiter if any
- Operations **`wait()`** and **`signal()`** must be executed atomically
- Semaphore types: Counting Semaphore, Binary Semaphore
- Binary – integer value can be 0 or 1 (known as mutex)
- Counting – integer value can be a range of values

Semaphores

Definition of the **wait()**operation

```
wait(S) {
    while (S <= 0);
     S=s-1; }
```

Definition of the **signal()**operation

```
signal(S) {
    S=s+1;
}
```

Semaphores

Mutex (Mutual exclusion) is a shared variable used by multiple process whose value is only binary as it is a binary semaphore and it is initialized with 1

Hence, the semaphore S in the wait and signal can be replaced by mutex

wait(mutex) {	signal(mutex) {
while (mutex <= 0);	mutex++;
mutex--; }	}

Semaphores


```
do {
Wait(mutex);
    //critical section
Signal(mutex);
  //remainder section
} while (TRUE);
```

OPERATING SYSTEM DESIGN

Producer-Consumer Problem

- Producer
  - adds items into the buffer
  - It has to stop when the buffer is full
  - It should not allow the consumer to consume while producing
- Consumer
  - Consumes an item from the buffer
  - It cannot consume from an empty buffer
  - It should not allow the producer to produce while consumer is consuming

Producer-Consumer Problem

PES UNIVERSITY
CELEBRATING 50 YEARS

Solution

- One semaphore to make these two process mutual exclusive
- One semaphore to maintain number of item in buffer
- One semaphore to maintain number of empty places in buffer

Producer

```
do  {
//   produce an item  A
wait (mutex);
wait (empty);
                  //  add the item
signal (full);
signal (mutex);
} while (TRUE);
```

Mutex	Empty	Full
1	3	0

```
Wait(mutex){
While(mutex<=0);
  mutex- -;
}
Signal(mutex){
 mutex++}
```

Consumer

```
do {
 wait (mutex);
wait (full);
              //  remove an item
signal (empty);
signal (mutex);

       } while (TRUE);
```

PES UNIVERSITY
CELEBRATING 50 YEARS

Producer-Consumer Problem

Producer

```
do  {
   //   produce an item in nextp
              wait (mutex);
              wait (empty);
                    //  add the item to the  buffer
              signal (full);
              signal (mutex);
} while (TRUE);
```

Producer-Consumer Problem

Consumer

```
do {
        wait (mutex);
       wait (full);
            //  remove an item from  buffer to nextc
     signal (empty);
       signal (mutex);
            //  consume the item in nextc
   } while (TRUE);
```

THANK YOU

S Thenmozhi

Department of Computer Applications

thenmozhis@pes.edu

+91 80 6666 3333 Extn 393